

CSCI 5380 - Network Virtualization and Orchestration

Lab 9

Automate VM, VN, Docker, and BGP path

University of Colorado Boulder
Department of Computer Science
Network Engineering

Professor Levi Perigo, Ph.D.

Summary:

In this lab, you will use what you have learned in previous labs and automate the processes into a single application.

Required technologies:

- BGP
- Hypervisor/Orchestrator (such as OpenStack)
- Containers
- SDN Controller
- Hardware server
- Service-chain

Objectives: Virtualized Network Automation

Create an application that meets the following functionality (each objective must be a separate Python module in your code i.e. your main .py file should import the different modules you write):

- 1) Automate the creation of multiple virtual networks (VNs) within the hypervisor and their connection to the public network.
- 2) Automate the creation of multiple VMs within the hypervisor-
 - a) Both single tenant (same VN) and multi-tenant (different VNs).
 - b) All VMs should be accessible from the host server and be able to access the Internet.
- 3) Automate the security groups and port security configuration to make intra-VN and inter-VN communication possible.
- 4) Automate spinning up and configuring a Quagga/FRR BGP router as a Docker container.
 - a) Automate its BGP configuration to peer with the SDN controller in the next objective.
- 5) Automate spinning up and configuring an SDN controller as another Docker container.
 - a) Automate its BGP speaker configuration to peer with Quagga/FRR.

Deliverable:

Create a personal GitHub page that demonstrates the required functionality.

I created an main script which imports all the modules and at the start it asks for option from user to configure the network and names based on which it configures the container and I have used FRR –

I am using clouds.yaml file to allow my scripts to interact with the openstack –

Creation of network in openstack –

```
NVO AUTOMATION LAB9 FRAMEWORK

[1] Create Virtual Network
[2] Add Virtual Machine
[3] Configure Security Groups
[4] Deploy FRR BGP Containers
[5] Deploy RYU Controllers
[6] Exit

Enter your choice: 1
1
2026-03-30 12:21:37.837 | INFO | __main__:main:30 - User selected option: 1
Enter network name: bgp
bgp
Enter subnet name: lab9
lab9
Enter router name: lab9
lab9
Enter CIDR: 10.10.10.0/24
10.10.10.0/24
Enter gateway IP: 10.10.10.1
10.10.10.1
2026-03-30 12:21:53.899 | INFO | __main__:main:39 - Creating VN: bgp
2026-03-30 12:21:55.383 | INFO | create_vn:create_virtual_network:12 - Created network: bgp
2026-03-30 12:21:56.848 | INFO | create_vn:create_virtual_network:24 - Created subnet: lab9
2026-03-30 12:21:57.213 | INFO | create_vn:create_virtual_network:32 - Created router: lab9
2026-03-30 12:21:59.240 | INFO | create_vn:create_virtual_network:37 - Router gateway set to public network
2026-03-30 12:22:01.555 | INFO | create_vn:create_virtual_network:40 - Attached subnet to router
```

We could verify the openstack UI and see the network and routers are created –

Displaying 3 items

☐	Name	Subnets Associated	Shared	External	Status	Admin State	Availability Zones	Actions
☐	shared	shared-subnet 192.168.233.0/24	Yes	No	Active	UP	-	
☐	bgp	lab9 10.10.10.0/24	No	No	Active	UP	-	Edit Network
☐	public	public-subnet 172.24.4.0/24 ipv6-public-subnet 2001:db8::/64	No	Yes	Active	UP	-	

Displaying 1 item

☐	Name	Status	External Network	Admin State	Availability Zones	Actions
☐	lab9	Active	public	UP	-	Clear Gateway

Once the network is created – I used option 2 to create an VM and mentioned same for network –

```
/home/atu/.virtualenvs/NVO/bin/python /home/atu/Labs/Lab 9/main.py

=====
 NVO AUTOMATION LAB9 FRAMEWORK
=====

[1] Create Virtual Network
[2] Add Virtual Machine
[3] Configure Security Groups
[4] Deploy FRR BGP Containers
[5] Deploy RYU Controllers
[6] Exit

Enter your choice: 2
2
2026-03-30 16:08:35.319 | INFO      | __main__:main:30 - User selected option: 2
Enter server name: lab9
lab9
2026-03-30 16:08:38.798 | INFO      | __main__:main:38 - Creating VM: lab9
2026-03-30 16:08:38.798 | INFO      | create_vm:get_internal_network_from_topology:28 - Using internal network from topology: bgp-net
2026-03-30 16:08:38.798 | INFO      | create_vm:create_server:44 - Starting server creation: lab9, bgp-net
2026-03-30 16:08:39.016 | INFO      | create_vm:create_server:56 - Image: b267a1bf-4563-43c2-8609-d28501da8edd, Flavor: c1, Network: ae5b8125-a948-49a5-b5d2-3a2268b74738
2026-03-30 16:08:39.270 | INFO      | create_vm:create_server:66 - Server lab9 created, waiting for ACTIVE state...
2026-03-30 16:08:54.631 | SUCCESS   | create_vm:create_server:68 - Server is ACTIVE: lab9
2026-03-30 16:08:54.632 | INFO      | create_vm:create_floating_ip:83 - Checking available floating IPs for lab9
2026-03-30 16:08:54.758 | WARNING   | create_vm:create_floating_ip:91 - No available floating IPs, creating a new one...
2026-03-30 16:08:55.414 | INFO      | create_vm:create_floating_ip:93 - Created new floating IP: 172.24.4.243
2026-03-30 16:08:56.607 | SUCCESS   | create_vm:create_floating_ip:102 - Attached Floating IP 172.24.4.243 to lab9
2026-03-30 16:08:56.608 | SUCCESS   | create_vm:create_server:73 - Server lab9 ready with IP: 172.24.4.243
```

Since the Network creation happened using a topology json file where name of the network and subnet were defined – the script found it from same place. We could also verify that the floating IP is also added so the instance will be able to ping each other.

```
login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
lab9 login: cirros
Password:
$
$
$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=111 time=3.90 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=111 time=3.40 ms
^C
```

Displaying 1 item

☐	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	lab9	cirros-0.6.3-x86_64-disk	192.168.50.120, 172.24.4.243	cirros256	-	Active	nova	None	Running	3 minutes	Create Snapshot

Displaying 1 item

Next steps are to create security group settings -

```
NVO AUTOMATION LAB9 FRAMEWORK

[1] Create Virtual Network
[2] Add Virtual Machine
[3] Configure Security Groups
[4] Deploy FRR BGP Containers
[5] Deploy RYU Controllers
[6] Exit

Enter your choice: 3
3
2026-03-30 16:11:33.993 | INFO      | __main__:main:30 - User selected option: 3
Enter security group: lab9
lab9
Enter server name: lab9
lab9
2026-03-30 16:11:41.774 | INFO      | __main__:main:46 - Creating security group: lab9
Created security group: lab9
Sending request to remove SG 'default' from server 'lab9'...
Successfully removed security group 'default' from server 'lab9'
2026-03-30 16:11:42.623 | INFO      | __main__:main:54 - === Security Group Services ===
2026-03-30 16:11:42.624 | INFO      | __main__:main:55 - 1. ICMP
2026-03-30 16:11:42.624 | INFO      | __main__:main:56 - 2. SSH
2026-03-30 16:11:42.624 | INFO      | __main__:main:57 - 3. TCP
2026-03-30 16:11:42.625 | INFO      | __main__:main:58 - 4. UDP
2026-03-30 16:11:42.625 | INFO      | __main__:main:59 - 5. Exit
2026-03-30 16:11:42.625 | INFO      | __main__:main:60 - =====
Enter your choice: |
```

Displaying 2 items

☐	Name	Security Group ID	Description	Shared	Actions
☐	default	dee34f56-2470-4c69-911a-dac2fbd67b11	Default security group	-	<button>Manage Rules</button>
☐	lab9	f9505125-5b6a-4678-8828-4e8c1c610744	Allow intra- and inter-VN traffic	-	<button>Manage Rules</button> ▼

We allowed ICMP –

```
Enter your choice: 1
1
2026-03-30 16:13:13.854 | INFO      | __main__:main:65 - Allowing ICMP
Added ICMP rule to security group: lab9
2026-03-30 16:13:14.405 | INFO      | __main__:main:54 - === Security Group Services ===
2026-03-30 16:13:14.405 | INFO      | __main__:main:55 - 1. ICMP
2026-03-30 16:13:14.406 | INFO      | __main__:main:56 - 2. SSH
2026-03-30 16:13:14.406 | INFO      | __main__:main:57 - 3. TCP
2026-03-30 16:13:14.406 | INFO      | __main__:main:58 - 4. UDP
2026-03-30 16:13:14.406 | INFO      | __main__:main:59 - 5. Exit
2026-03-30 16:13:14.406 | INFO      | __main__:main:60 - =====
```

We also allowed SSH rules –

```
2026-03-30 16:29:09.269 | INFO | __main__:main:30 - User selected option: 3
Enter security group: lab9
lab9
Enter server name: lab9
lab9
2026-03-30 16:29:14.162 | INFO | __main__:main:46 - Creating security group: lab9
2026-03-30 16:29:14.257 | INFO | create_security_groups:get_or_create_security_group:23 - Using existing security group: lab9
2026-03-30 16:29:14.592 | INFO | create_security_groups:detach_security_group:49 - Security groups currently attached to lab9: ['1', 'lab9']
2026-03-30 16:29:14.592 | INFO | create_security_groups:detach_security_group:52 - Security group 'default' not attached to server 'lab9', skipping removal.
2026-03-30 16:29:14.592 | INFO | __main__:main:54 - === Security Group Services ===
2026-03-30 16:29:14.592 | INFO | __main__:main:55 - 1. SSH
2026-03-30 16:29:14.592 | INFO | __main__:main:56 - 2. TCP
2026-03-30 16:29:14.592 | INFO | __main__:main:57 - 3. UDP
2026-03-30 16:29:14.593 | INFO | __main__:main:58 - 4. ICMP
2026-03-30 16:29:14.593 | INFO | __main__:main:59 - 5. Exit
2026-03-30 16:29:14.593 | INFO | __main__:main:60 - =====
Enter your choice: 1
1
2026-03-30 16:29:16.696 | INFO | __main__:main:65 - Allowing SSH
2026-03-30 16:29:16.732 | INFO | create_security_groups:add_ssh_rule:144 - SSH rule already exists in security group: lab9
2026-03-30 16:29:16.987 | INFO | create_security_groups:attach_sg_if_not_attached:108 - Security group 'lab9' already attached to server 'lab9'.
2026-03-30 16:29:16.988 | INFO | __main__:main:54 - === Security Group Services ===
2026-03-30 16:29:16.988 | INFO | __main__:main:55 - 1. SSH
2026-03-30 16:29:16.988 | INFO | __main__:main:56 - 2. TCP
2026-03-30 16:29:16.989 | INFO | __main__:main:57 - 3. UDP
2026-03-30 16:29:16.989 | INFO | __main__:main:58 - 4. ICMP
2026-03-30 16:29:16.989 | INFO | __main__:main:59 - 5. Exit
2026-03-30 16:29:16.989 | INFO | __main__:main:60 - =====
Enter your choice:
```

Manage Security Group Rules: lab9 (f9505125-5b6a-4678-8828-4e8c1c610744)

+ Add Rule

Delete Rules

Displaying 3 items

☐	Direction	Ether Type	IP Protocol	Port Range	Remote IP Prefix	Remote Security Group	Description	Actions
☐	Egress	IPv4	Any	Any	0.0.0.0/0	-	-	Edit Rule
☐	Egress	IPv6	Any	Any	:::0	-	-	Edit Rule
☐	Ingress	IPv4	ICMP	Any	0.0.0.0/0	-	-	Edit Rule

Displaying 3 items

The script handles if rule is already there then skip it.

FRR instance is run and I have config file which is used to mounted in FRR to run it –

```
/home/atuL/.Virtualenvs/NVO/bin/python /home/atuL/Labs/Lab 9/main.py

NVO AUTOMATION LAB9 FRAMEWORK

[1] Create Virtual Network
[2] Add Virtual Machine
[3] Configure Security Groups
[4] Deploy FRR BGP Containers
[5] Deploy RYU Controllers
[6] Exit

Enter your choice: 4
4
2026-03-30 18:28:27.853 | INFO | __main__:main:30 - User selected option: 4
2026-03-30 18:28:27.854 | INFO | __main__:main:87 - Starting FRR deployment using deployment.json
2026-03-30 18:28:27.854 | INFO | create_frr:load_config:15 - Loaded deployment configuration
2026-03-30 18:28:27.854 | INFO | __main__:main:92 - Starting FRR deployment using topology.json - network: bgp-net router: {'name': 'frr', 'ip': '192.168.60.2', 'config_path': './configs/frr', 'subnet': '192.168.60.0/24', 'ga
2026-03-30 18:28:27.856 | INFO | create_frr:ensure_network:25 - Network 'bgp-net' already exists
2026-03-30 18:28:28.883 | SUCCESS | create_frr:run_router:93 - Router 'frr' running at 192.168.60.2
2026-03-30 18:28:28.883 | SUCCESS | __main__:main:94 - FRR deployment completed
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
ef56f54288d8	frrouting/frr:latest	"/sbin/tini -- /usr/..."	About a minute ago	Up About a minute	0.0.0.0:6769->6769

BGP summary –

```
bash-5.1# vtysh
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.

Hello, this is FRRouting (version 8.4_git).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

ef56f54288d8# show ip bgp summary

IPv4 Unicast Summary (VRF default):
BGP router identifier 1.1.1.1, local AS number 100 vrf-id 0
BGP table version 0
RIB entries 1, using 192 bytes of memory
Peers 1, using 717 KiB of memory

Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down State/PfxRcd  PfxSnt Desc
192.168.60.3   4      200        0        0        0    0    0   never      Active        0 N/A

Total number of neighbors 1
```

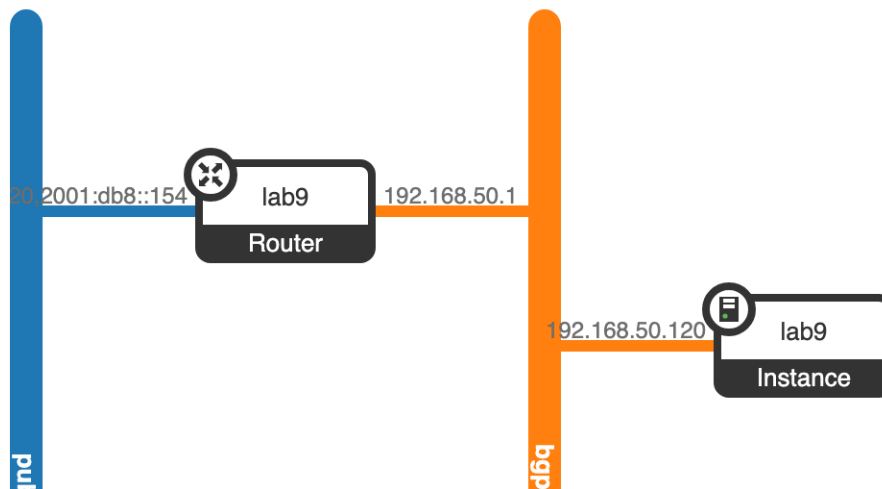
Option 5 is RYU controller deployment –

```
2026-03-30 19:28:54.955 | INFO      | controller:run_ryu_controller:28 - Starting RYU controller: ryu
loading app bgp_app.py
Create CustomEvent called
instantiating app bgp_app.py of FinalDynamicController
API method core.start called with args: {'router_id': '3.3.3.3', 'label_range': (100, 100000), 'waiter': <ryu
Create CustomEvent called
Starting new processing run...
Processing RT NLRI destination...
Processing destination...
API method neighbor.create called with args: {'connect_mode': 'both', 'cap_mbgp_evpn': False, 'remote_as': 100
Create CustomEvent called
Create CustomEvent called
API method network.add called with args: {'prefix': '200.1.1.0/24'}
SIGNAL: ('core', 'dest', 'changed') emitted with data: Destination(table: Ipv4Table(scope_id: None, rf: Route
[SYSTEM] BGP Speaker initialized. Re-Sync logic active.
BRICK FinalDynamicController
Started peer Peer(ip: 192.168.60.2, asn: 100)
start connect loop. (mode: both)
Starting new processing run...
Processing RT NLRI destination...
Processing destination...
Processing destination: Destination(table: Ipv4Table(scope_id: None, rf: RouteFamily(afi=1,safi=1)), nLri: IP
```

It uses same network as FRR instance on docker network to ensure it can directly talk to the FRR container to establish BGP.

From FRR we could see the connectivity as well –

```
PING 192.168.60.3 (192.168.60.3): 56 data bytes
64 bytes from 192.168.60.3: seq=0 ttl=64 time=0.127 ms
64 bytes from 192.168.60.3: seq=1 ttl=64 time=0.074 ms
64 bytes from 192.168.60.3: seq=2 ttl=64 time=0.108 ms
64 bytes from 192.168.60.3: seq=3 ttl=64 time=0.072 ms
64 bytes from 192.168.60.3: seq=4 ttl=64 time=0.079 ms
```



FRR and RYU on docker container –

```

atu1@csci5380-vm2-atan8167:~$ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
5f2d9a3a01ca	osrg/ryu	"tail -f /dev/null"	5 minutes ago	Up 5 minutes	
ef56f54288d8	frouting/frr:latest	"/sbin/tini -- /usr/..."	About an hour ago	Up About an hour	